

SPANS-99: Best Practice Summary

Series: SPANS — Distributed Tracing and Spans | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 10/02/2026

Definitive best practice settings for distributed tracing and span analysis. Each entry specifies the exact configuration.

Table of Contents

- 1. [DQL Syntax Rules](#)
- 2. [Filtering & Performance](#)
- 3. [Time Range & Cost](#)
- 4. [Duration & Aggregation](#)
- 5. [Trace Analysis](#)
- 6. [Service Dependencies](#)
- 7. [Security](#)
- 8. [OpenPipeline Span Configuration](#)
- 9. [Bucket Strategy & Sampling](#)

1. DQL Syntax Rules

Practice	Recommended Setting/Value	Priority
Equality operator	<code>==</code> (double equals) — never <code>=</code>	Critical
Array literals	<code>{"a", "b"}</code> with curly braces and double quotes — never <code>('a', 'b')</code>	Critical

Practice	Recommended Setting/Value	Priority
NULL checks	<code>isNull(field) / isNotNull(field) — field == null</code> is rejected (" <i>null isn't allowed here</i> ")	Critical
Multi-value matching	<code>in(field, {"val1", "val2"})</code> — never SQL <code>IN</code>	Critical
<code>span.kind</code> values	Lowercase only: <code>"server"</code> , <code>"client"</code> , <code>"internal"</code> , <code>"producer"</code> , <code>"consumer"</code>	Critical
<code>span.status_code</code> values	Lowercase <code>"error"</code> / <code>"ok"</code> ; the field is absent when unset (most spans), so <code>!= "error"</code> does not count successes — use total minus errors	Critical
String literals	Double quotes everywhere — never single quotes	Critical
Grouping syntax	<code>summarize n = count(), by:{field}</code> with curly braces — never <code>GROUP BY</code>	Critical
Service field	<code>dt.service.name</code> (every span) — not <code>service.name</code> (OpenTelemetry spans only; 6.7% on the validation tenant). Joins and identity: <code>dt.smartscape.service</code> (<code>dt.entity.service</code> is deprecated) — names can repeat across applications	Critical
ID fields are <code>uid</code>	<code>trace.id == toUId("...")</code> — a plain string comparison is always false and returns zero rows	Critical
Always alias aggregation results	<code>summarize error_count = count()</code> — required for <code>sort</code> and <code>fieldsAdd</code>	Critical

2. Filtering & Performance

Practice	Recommended Setting/Value	Priority
Filter immediately after fetch	All <code>filter</code> commands before any <code>fieldsAdd</code> , <code>summarize</code> , <code>sort</code>	Critical
Selective filters first	Grail has no field indexes ; a filter that rules out most data lets it skip reading (1 h: no filter 910 MB, <code>dt.service.name == "checkout"</code> 54 MB, <code>trace.id == toUId(...)</code> 57 MB)	Critical

Practice	Recommended Setting/Value	Priority
Measure, don't guess	Read <code>scannedBytes</code> in the query result before and after a change	Recommended
<code>==</code> for exact matches, <code>~</code> for wildcards only	<code>field == "exact"</code> is faster than <code>field ~ "exact"</code>	Recommended
Combine conditions in single filter	<code>filter span.kind == "server" and dt.service.name == "checkout" and duration > 100ms</code>	Recommended
Substring tests after selective filters	<code>contains(url.path, ...)</code> skips little data on its own	Recommended
Select only needed fields	Smaller results — scanned bytes are unchanged	Optional
Compute derived fields after filtering	<code>fieldsAdd duration_ms = ...</code> must appear after all filter steps	Recommended
Sort after summarize, never after fetch	Sorting before aggregation wastes resources	Critical
Limit after sort, at pipeline end	<code>sort desc \ limit 20</code> as final commands — never <code>limit</code> before <code>summarize</code>	Critical

3. Time Range & Cost

Practice	Recommended Setting/Value	Priority
Always specify explicit time range	<code>fetch spans, from:-1h</code> — never bare <code>fetch spans</code>	Critical
Use narrowest range possible	Debugging: <code>from:-5m</code> . Recent: <code>from:-1h</code> . Daily: <code>from:-24h</code> . Weekly: <code>from:-7d</code> (1 h → 15 min cut 910 MB to 234 MB)	Critical
Target specific buckets	<code>fetch spans, from:-1h, bucket:{"<your bucket>"}</code> — check <code>dt.system.bucket</code> first	Recommended

Practice	Recommended Setting/Value	Priority
Read sampling for estimates	<code>fetch spans, samplingRatio: 100</code> then multiply counts by <code>dt.system.sampling_ratio</code> (the read-sampling ratio, not the trace sampler)	Optional
Never group by <code>trace.id</code> without pre-filtering	Millions of unique values — filter by error or time first	Critical
Group by low-cardinality fields for dashboards	<code>dt.service.name</code> , <code>span.name</code> , <code>span.kind</code> , <code>http.route</code> — never <code>url.path</code> , <code>trace.id</code>	Recommended
Always <code>limit</code> non-aggregated queries	<code>limit 100</code> on any raw span query	Critical
Use only needed percentiles	Each adds compute time, not scanned bytes	Optional

4. Duration & Aggregation

Practice	Recommended Setting/Value	Priority
Convert duration to a number	<code>duration / 1ms</code> , <code>duration / 1s</code> — never <code>/ 1000000</code> (still a duration)	Critical
Use duration literals in filters	<code>filter duration > 100ms</code> — an integer literal never matches a duration	Critical
Use <code>countIf()</code> for conditional counts	<code>summarize errors = countIf(span.status_code == "error"), total = count()</code> in single pass	Recommended
Error rate calculation	<code>(error_count * 100.0) / total_requests</code> — use <code>100.0</code> (float) to avoid integer division	Recommended

Practice	Recommended Setting/Value	Priority
Percentile trends	<code>makeTimeseries p95_ms = percentile(duration / 1ms, 95), interval:10m</code> — conversion inside the aggregation; <code>percentile(duration, 95) / 1ms</code> is rejected	Critical
Use <code>makeTimeseries</code> for dashboard charts	<code>makeTimeseries {requests = count(), errors = countIf(span.status_code == "error")}, interval:5m</code>	Recommended
Server error rate	Count <code>span.status_code == "error"</code> or <code>http.response.status_code >= 500</code> — 7.7% of 5xx server spans carried no status on the validation tenant	Recommended
Minimum sample size before ranking	<code>filter request_count > 10</code> before percentile/error rate rankings	Recommended

5. Trace Analysis

Practice	Recommended Setting/Value	Priority
Find root spans	<code>filter isNull(span.parent_id)</code>	Recommended
Reconstruct trace chronologically	<code>filter trace.id == toUid("ID") \ sort start_time asc</code>	Recommended
Find the ORIGIN error in a trace	The error span that ends first : <code>summarize origin = takeMin(record(end_time = end_time, service = dt.service.name, span = span.name)), by:{trace.id}</code> . The earliest- <i>starting</i> error is usually a caller that inherited it (231 of 247 multi-error traces)	Critical
Trace duration	<code>(max(end_time) - min(start_time)) / 1ms</code> per <code>trace.id</code> — not <code>sum(duration)</code> (double-counts nested work)	Recommended

Practice	Recommended Setting/Value	Priority
Detect cascading failures	<code>filter span.status_code == "error" \</code> <code>summarize n = count(), by:{trace.id} \</code> <code>filter n > 1</code>	Recommended
Impact score for bottleneck spans	<code>fieldsAdd impact_score = call_count * avg_duration_ms \</code> <code>sort impact_score desc</code>	Recommended
<code>sum(duration)</code> per service is inclusive time	It includes child calls a span waits on — rank by it, don't call it self time	Recommended
Health scorecard classification	<code>if(error_rate > 5, "Critical", else:</code> <code>if(error_rate > 1, "Warning", else:</code> <code>"Healthy"))</code>	Recommended

6. Service Dependencies

Practice	Recommended Setting/Value	Priority
Map dependencies via CLIENT spans	<code>filter span.kind == "client" \</code> <code>summarize n = count(), by:{dt.service.name,</code> <code>server.address}</code>	Recommended
Caller → callee service map	Join server spans to the client span named in <code>span.parent_id</code> (same <code>trace.id</code>) — see SPANS-04 §3	Recommended
Don't rely on <code>peer.service</code>	No semantic-dictionary row; 0 of 8.29M spans in 24 h on the validation tenant	Critical
Track async messaging	<code>filter span.kind == "producer" or</code> <code>span.kind == "consumer" \</code> <code>summarize n = count(), by:{messaging.system,</code> <code>messaging.destination.name}</code> <code>(messaging.operation.type , not</code> <code>messaging.operation)</code>	Recommended
Outbound/inbound ratio	<code>toDouble(outbound) / toDouble(inbound)</code> per <code>dt.service.name</code> — integer division truncates	Optional

7. Security

Practice	Recommended Setting/Value	Priority
Use <code>http.route</code> in dashboards, never <code>url.path</code>	<code>http.route</code> = pattern (no PII), <code>url.path</code> = actual values (may contain PII)	Critical
Audit query strings where they live	Tokens sit in <code>url.query</code> / <code>url.full</code> , not <code>url.path</code> — check all three	Critical
Mask card numbers as whole path segments	<code>replacePattern(url.path, "<< '/' CREDITCARD (>>'/' \ >>'?' \ EOF)", "[CARD-MASKED]")</code> — a bare <code>CREDITCARD</code> also matches digit runs inside hex IDs	Critical
Weekly PII audit: emails in URLs	Leak check: <code>countIf(replacePattern(url.path, "[A-Za-z0-9._%+-]+ '@' [A-Za-z0-9.-]+", "")) != url.path)</code> — must be 0	Critical
Weekly audit: sensitive data in <code>db.query.text</code>	<code>contains(db.query.text, "password", caseSensitive: false)</code> and similar (<code>db.statement</code> is the null pre-1.0 name)	Critical
Exceptions are span events	<code>iAny(span.events[][span_event.name] == "exception"); text in exception.message / exception.stack_trace</code>	Recommended
Monitor 401/403/429 as security signals	<code>filter in(http.response.status_code, {401, 403, 429})</code> with 10-min bucketing on dashboard	Critical
Brute force detection	401 count >10 per hour per endpoint = alert (community threshold — tune it)	Recommended
Enumeration detection	404 count >100 with high unique path count = scanning alert (community threshold)	Recommended
Limit <code>trace.id</code> exposure	<code>takeAny(trace.id)</code> per error pattern — never expose all trace IDs	Recommended

Practice	Recommended Setting/Value	Priority
Mask emails in span URLs via OpenPipeline	DQL processor: <code>replacePattern(url.path, "[A-Za-z0-9._%+-]+ '@' [A-Za-z0-9.-]+", "[EMAIL-MASKED]")</code>	Critical
Mask tokens in query params via OpenPipeline	DQL processor: <code>replacePattern(url.query, "<<(('token' \\ 'key' \\ 'secret' \\ 'password') '=') [^&]+", "[REDACTED]")</code> — DPL, no back-references	Critical

8. OpenPipeline Span Configuration

Practice	Recommended Setting/Value	Priority
Drop health check spans at ingestion	Drop record processor, matching condition <code>matchesValue(span.name, "*health*")</code> or <code>matchesValue(span.name, "*/ready*")</code> (matchers reject <code>contains()</code>)	Critical
Drop static asset spans	Drop record on <code>matchesValue(url.path, "*.js")</code> or <code>matchesValue(url.path, "*.css")</code> — measure first	Recommended
Never drop error or slow spans	Leave <code>span.status_code == "error"</code> and <code>duration > 1s</code> out of every drop condition	Critical
Pre-compute <code>duration_ms</code> at ingestion	DQL processor: <code>fieldsAdd duration_ms = duration / 1ms</code>	Optional
Route to environment buckets	Bucket assignment (first match): production → long-retention bucket, staging → short, <code>true</code> last	Recommended
Route sensitive service spans to restricted buckets	Payment/auth services → <code>spans_sensitive</code> with restricted IAM	Recommended
Bucket-level IAM for team isolation	Frontend team: <code>spans_frontend</code> only. SRE: all span buckets. Compliance: <code>audit_spans</code>	Recommended

9. Bucket Strategy & Sampling

Practice	Recommended Setting/Value	Priority
Retention by environment	Production: 90 days. Staging: 7 days. <code>default_spans</code> is built in (10 days on the validation tenant); Grail traces are <i>"Configurable, from 10 days to 10 years"</i>	Recommended
Sample at the source, not in OpenPipeline	OneAgent: Adaptive Traffic Management (head-based). OpenTelemetry: Collector <code>tail_sampling</code> — the only place "keep all errors" is possible. <i>"OpenPipeline processing cannot be used as an alternative to ATM configuration for controlling trace volume."</i>	Critical
Compute metrics before tail sampling	<code>spanmetrics</code> connector ahead of <code>tail_sampling</code> in the Collector	Recommended
Calculate savings before filtering	Count health checks + static assets (<code>url.path</code>) vs errors + slow to size droppable vs must-keep	Recommended
Extract <code>dt.ingest.size</code> as metric	Sampling aware value metric (the value-metric variant on spans), key <code>span.ingest.size.by.app</code> , dim: app identifier	Recommended
Verify bucket distribution monthly	<code>summarize n = count()</code> , by: <code>{dt.system.bucket}</code> — verify routing rules working	Recommended

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.